

ACSD: Authorized Lineage for Anonymous Scholarly Releases

Anonymous authors

Abstract

Anonymous scholarly releases may change authors, acquire new versions, and disclose selected identities or research records after publication. Signatures, timestamps, and archives supply essential building blocks, but their composition must specify who authorizes each transition, which signatures existed by a given time, and what a later disclosure establishes. We present ACSD, an executable protocol profile for these decisions. A jointly signed target binds the release, author governance, evidence policy, and exact lineage transition. A separate approval set closes over the completed signatures before timestamping. Predecessor authority controls succession, including a precommitted recovery path; typed evidence rules constrain partial disclosure and derived claims. Lean proves target and transition non-reuse, required authorization, and scope non-amplification, with executable transcript checkers connecting verified adapter facts to the model. Evaluation combines adversarial lifecycle cases, independent implementation checks, a scripted three-author coordination run, and 90 measured local creation/verification samples comparing ACSD, detached Ed25519, and actual GnuPG. A manual signed transition is a successful authorization control. ACSD adds explicit composition rules and their verification at measurable storage and runtime cost. The resulting evidence supports pseudonymous assent and authorized evolution; historical authorship and the truth of scientific contributions require separate assessment.

1 Introduction

Researchers often want to disclose a manuscript before conventional publication while keeping the author identity unavailable during review. A common informal proposal is to upload an anonymous file, sign it, and rely on repository history. That proposal leaves several distinct questions entangled.

One concrete trigger is the interval around an arXiv submission: a first-time or new-category submission may require endorsement, while moderation and review are separate from scientific peer review. During that interval an author may want a public, content-anonymous evidence package that preserves exact bytes and team assent without pretending to replace arXiv’s process. We use this as an application scenario, not as a claim that arXiv queues have a measured backlog or that ACSD should circumvent its rules.

Why signatures and timestamps do not specify the workflow. Consider three pseudonymous authors who release a manuscript, replace one team member in its next version, and later disclose only two names. GPG can verify each signature, OpenTimestamps can witness a committed object’s time, and Zenodo can archive a release. These operations do not specify whether the old team authorized the replacement, whether the time evidence covers every required approval, or whether two disclosures justify a full-byline claim. An attacker can sign an apparent next version under a fresh key and obtain valid time evidence for it while none of those operations establishes predecessor authorization. The missing contract is the relation among signed objects, authorized transitions, and permitted conclusions.

ACSD (Anonymous Scholarly Claim and Disclosure) makes that contract explicit. The authoring path binds a release, its governance statement, its *Provenance Evidence Capsule* (PEC), and an exact successor transition to one approval target. Verification requires the applicable author authority and checks each disclosed claim against evidence of the correct type and scope. An independent RFC 3161 timestamp can then cover the completed signature set. The public content may omit names while hosting accounts and reused keys remain linkable.

Ordinary cryptographic components can implement these rules. Our manual predecessor-signature control confirms that an explicitly constructed transition can reproduce the core succession decision. That control tests the effect of adding the authorization rule; it is not functionality supplied automatically by detached signing or archival publication. The contribution is the protocol design, its attack-exclusion properties, and an executable implementation.

Contributions.

1. **An explicit composition protocol.** Joint approval binds manuscript, governance and evidence policy; a second acyclic object binds completed signatures before timestamping. Exact predecessor authorization governs revisions and precommitted recovery. Optional series objects allow mutual citation without circular content hashes.
2. **Formal attack-exclusion results.** Lean proves that approvals and transitions cannot be reused for different targets, accepted successors require the applicable authority, and evidence cannot grant claims beyond its permitted type and exact scope. Executable transcript checking and refinement connect these rules to adapter outputs. These statements quantify over the modeled inputs, beyond the finite adversarial cases exercised by tests.
3. **An evaluated reference implementation.** A command-line workflow supports team approval, authorized revision, recovery, distributed approval collection, exact-target delegation to a coordinator agent, and partial/full identity disclosure. Independent verifiers, adverse transcripts, a successful manual authorization control, actual GnuPG comparisons and a real timestamp receipt make its behavior and costs reproducible.

2 Architecture and Evidence Flow

Figure 1 separates object construction, external evidence checking, and the formal claim boundary. The release digest fixes manuscript bytes and version metadata. Governance fixes author roles and declarations. The PEC commits evidence events and policy. For a successor, the transition also fixes the parent, child and proposed authority. All required keys approve the same target; finalization then fixes the exact signature set. These two stages distinguish assent to an object from evidence that the completed assent existed by a particular time.

Later timestamp receipts, dialogue openings and identity disclosures are separately checked against their original subjects. The adapter checks produce facts for the claim layer. The Lean model proves how those facts may compose, and the strict transcript checker reconstructs the corresponding derivations. The adapter boundary and model assumptions are explicit; Section 9.3 states the principal security properties.

3 Scope and Threat Model

Assets. A release contains manuscript bytes, an ordered list of anonymous author keys, roles and contribution declarations, an AI-use declaration, and optional parent-release information. A

series package contains references to completed releases, a series key, sequencing information, and cross-paper semantic relations. Public keys are pseudonymous identifiers.

Adversary. We consider a party that can copy public content, generate fresh keys, reuse a WorkID and predecessor digests, or claim the next unused version. The party may reorder or substitute unsigned metadata, timestamp a target before adding signatures, replay a disclosure or transition under another scope, substitute an uncommitted recovery guardian, combine ordinary and recovery authorizations, or present a partial byline as complete. It may also reuse a key across works, submit signed but malformed JSON, claim unsupported cross-paper citations, or cause a series key to sign incompatible objects. Finally, a malicious or compromised author-slot key may sign a false identity string naming an uninvolved person. We assume neither that signatures reveal human identities nor that contribution declarations are true, recovery guardians remain honest or available, or local timestamps are trusted.

Non-goals. ACSD does not prove authorship by a natural person, independent discovery, plagiarism, venue acceptance, peer review, or legal nonrepudiation. These are not merely unimplemented features: some require social, institutional, or legal evidence outside the cryptographic object. The profile instead preserves evidence that can be useful in a later dispute.

4 Security Goals and Trust Boundary

The protocol is designed around four acceptance properties rather than a generic claim that a signature proves authorship. Let $A(x, E)$ mean that the profile accepts object or conclusion x from evidence set E .

G1: successor-authorization soundness. If a child release is accepted as the successor of a given predecessor, E must contain an ordinary authorization by the applicable predecessor authority or an authorization by the exact recovery authority committed in advance. Knowing the WorkID, choosing the next version number, and signing under a fresh key are insufficient.

G2: exact-transition non-reuse. Authorization for a transition binds its parent, child, proposed authority and transition mode. Evidence accepted for one child cannot authorize a distinct child, even when both children reuse the same visible slot metadata.

G3: scope and type preservation. Every derived conclusion must be permitted by the closed policy and supported by verified evidence of the required type for the exact subject. In particular, one author’s identity statement cannot disclose another slot, an event opening cannot establish identity, and a partial byline cannot silently become a full byline.

G4: completed-assent time integrity. Time evidence over an unsigned target establishes, at most, target existence. The stronger conclusion that the required approvals existed by a stated time requires evidence over the closed approval-set object and the applicable external-time checks.

These properties assume collision-resistant object identifiers, unforgeable signatures for uncompromised keys, correct results from the explicitly named adapters, and faithful evaluation of the model’s predicates. The Lean development proves composition and attack exclusion above those inputs; it does not re-prove cryptographic primitives, general PKIX validation, filesystem integrity, or the honesty of a timestamp authority. Compromised authorized keys can issue protocol-valid

false statements within their scope. This boundary is intentional: the system makes such residual authority visible rather than translating it into an unsupported natural-person or originality claim.

5 Release Layer

5.1 Canonical release object

Each *PaperRelease* has a random stable *WorkID*, a visible slot (*WorkID*, *version*, *line*), a content path, content length and digest, ordered author records, declared roles, contributions, an AI-use declaration, optional exact predecessor, and a list of citation witnesses. The release identifier is

$$\text{ReleaseID} = \text{urn:sha256:H}(\text{canonical}(R)).$$

The content digest is over exact manuscript bytes. The release object uses a deliberately restricted JSON encoding: recursively sorted ASCII object keys, no insignificant whitespace, and safe integers only. A verifier parses, re-canonicalizes, and compares bytes before accepting a signature. This is a profile rule, not a claim that it is a general JSON standard.

5.2 Joint approval target and closed capability calculus

The *AuthorshipGovernanceStatement* is the exact declaration of the work id, manuscript digest, ordered byline slots and roles, corresponding-author key, and AI-use field. The reference builder derives roles from the team configuration and release; there is no privileged coordinator whose unsigned word makes it authoritative. Instead, every declared author key approves its digest through the common target described below. This proves assent to the declaration, not the factual truth of its roles or contribution history.

The command-line builder accepts repeatable AI tool and purpose declarations and one-based author slots recording human review. It rejects a partial declaration that supplies a tool without a purpose or vice versa. These fields, like contribution terms, are approved inside the common target. They remain author assertions: the package preserves what the team approved but does not independently establish that the declaration is exhaustive or factually true. Fine-grained links from selected dialogue events to particular version changes remain a separate disclosure workflow.

A *Provenance Evidence Capsule* (PEC) binds the release subject and governance digest to a digest-linked evidence-event sequence and a claim policy. The policy contains a fixed-vocabulary list of permitted outcomes, their required capabilities, and mandatory non-claims. A PEC also names one team key as an issuer for provenance, but that label alone grants no outcome: the complete team must still approve the common target.

The current command-line profile constructs an acyclic **acsd-approval-target/v2** object. It points to already complete objects and is itself signed only through detached approval envelopes. The following abbreviated schema excerpt shows its fields; angle-bracketed values are presentation placeholders rather than an accepted encoding:

```
{
  "schema": "acsd-approval-target/v2",
  "work_id": "urn:uuid:<work-id>",
  "release_digest": "<sha256>",
  "governance_digest": "<sha256>",
  "pec_digest": "<sha256>",
  "required_key_ids": [<key-id>],
  "lineage_transition_digest": null
}
```

Every required key issues a tagged COSE Sign1 signature over the same canonical target bytes. Verification recomputes all four bindings and each key id from the supplied public key. The closed capability calculus is the rule

$$\begin{aligned} \text{Granted}(P, o) \iff & \text{Accepted}(P) \\ & \wedge o \in P.\text{permittedOutcomes} \\ & \wedge \text{CapabilityEstablished}(P, o). \end{aligned}$$

Policy permission is therefore only an upper bound; it is not evidence. The outcome vocabulary is closed and versioned, so an unknown social claim cannot enter the rule by adding a string to JSON.

Without the joint target, suppose authors sign only the release while an attacker later changes the unsigned governance object or PEC policy. The attacker could relabel roles, add the external-time outcome, or try to map a local clock to that outcome while the old release signatures still verify. Under ACSD, changing the governance or policy changes its digest and causes a target-binding failure; recomputing the target instead causes the old COSE approvals to fail their payload check. Even an approved policy entry yields no outcome without its separately verified capability. A verifier can thus state that the pseudonymous keys jointly approved this exact tuple, but cannot infer who controls them without separate enrollment evidence.

The approval target is deliberately unsigned data until its detached signatures arrive. Timestamping it at that stage would establish only *target existence*: an attacker who later obtains a key could append a valid approval after the recorded time. Finalization therefore constructs an `acsd-approval-set/v1` object containing the target digest and the SHA-256 digest of every exact author-approval COSE byte string, plus every separate predecessor-authorization signature used for an authority change. Only this closed approval-set object is the timestamp subject. Replacing, adding, or removing a signature changes the approval-set digest. This is an evidence closure rule, not a new signature or timestamp primitive.

5.3 Authorized succession rather than version-number possession

An exact predecessor digest proves structural continuity, not authority. A fresh-key attacker can copy a public WorkID, name the real predecessor, choose the consecutive version number, and validly sign the resulting child with the attacker's keys. No hash or slot-conflict check answers whether those keys were entitled to create the parent-to-child edge. The current profile therefore binds every release to a *lineage authority*: a sorted author-key set and a threshold for authorizing the next edge. The default threshold is unanimity. It is stored in the predecessor, so a child cannot lower it unilaterally.

Let P be an accepted predecessor, C a proposed child, A_P the authority bound by P , and A_C the authority proposed by C . The verifier accepts a lineage edge only if

$$\begin{aligned} \text{AuthorizedSuccessor}(P, C) = & \text{ExactEdge}(P, C) \\ & \wedge \text{Authorization}(P, C). \end{aligned}$$

For unchanged authority, authorization is

$$\text{Authorization}(P, C) = \text{Quorum}_P(\text{ApproveTarget}(C)),$$

where Quorum_P applies the author set and threshold fixed by P . For changed authority, let $T_{P,C} = \text{Transition}(P, C)$; then

$$\begin{aligned} \text{Authorization}(P, C) = & (\text{Quorum}(A_P, T_{P,C}) \\ & \vee \text{RecoveryQuorum}(R_P, T_{P,C})) \\ & \wedge \text{AllApprove}(A_C, C). \end{aligned}$$

Here `ExactEdge` requires the same `WorkID`, exact parent release and PEC digests, and either a same-line increment by one or a new named line beginning at version one. `Transition` binds both authorities and the exact child release, governance, and PEC digests. The child approval target binds the transition digest. The construction is acyclic because the transition refers to complete child bodies while detached approvals refer to their completed digests.

For an unchanged authority object, the ordinary child approvals already constitute predecessor authorization and no duplicate signing step is needed. A key-set or threshold change uses double control: the predecessor threshold signs the exact transition and every new author signs the child target. Here R_P is optional: the predecessor may commit a sorted, nonempty recovery-key set disjoint from A_P , together with its threshold. A recovery quorum can replace—not join—the online predecessor quorum only for an authority-changing child, and every child author still approves the exact child target. This resembles the old-and-new authorization pattern used for TUF root-key updates, but ACSD applies it to a single scholarly lineage edge rather than software repository metadata [15]. A series key may advertise a verified child as a head; it cannot manufacture predecessor-author assent by itself.

Consider the concrete $n+1$ attack. Mallory constructs a child with P’s `WorkID`, P’s exact release and PEC digests, the next version number, Mallory’s public key, and a valid Mallory signature over every child-controlled object. The object is internally signed but has neither an unchanged predecessor authority nor a transition signed by A_P . The implementation returns `VALID_OBJECT_BUT_UNAUTHORIZED_SUCCESSOR`. It does not accuse Mallory of plagiarism; it rejects only the claimed lineage edge.

This conclusion is relative to an accepted exact predecessor, not to the `WorkID` string alone. The CLI can pin a previously accepted parent with `-expected-parent-release-id`; otherwise it reports `UNPINNED_EXACT_PARENT`. Without an external parent anchor, an isolated package proves only that the authority named by its embedded parent authorized the edge; it cannot distinguish a separately fabricated genesis that copied a public `WorkID`.

Threshold choice exposes a real safety–liveness tradeoff. Unanimity prevents one coauthor from transferring the lineage alone but lets one lost or dissenting key freeze it. A lower threshold improves availability and is therefore allowed only when committed by the predecessor. A recovery set is a second, prospective tradeoff: it is public and linkable, must be committed before loss, and can authorize one exact authority-changing transition, but it cannot create a post-hoc identity recovery rule or revoke a valid past authorization. Without a precommitted recovery authority, lost threshold keys safely stop the cryptographic lineage. Two valid distinct children in the same exclusive slot, including one online-authorized child and one recovery-authorized child, are reported as equivocation with no automatic winner; choosing by observation time requires an external transparency or governance policy.

5.4 Citation witnesses

The prototype recognizes only literal occurrences of a canonical token `urn:uuid:<work-id>` in the UTF-8 manuscript bytes. For every occurrence it records and signs

(`WorkID`, *byteOffset*, *byteLength*).

The published `reference_work_ids` list must equal the ordered de-duplicated projection of these witnesses. This prevents a series package from inventing a citation edge that has no corresponding token in its source release, and prevents a copied text from silently relabeling an old target as a new `WorkID`.

The rule intentionally does not parse PDFs, infer references from natural language, or determine whether a citation is intellectually appropriate. It is an exact byte-binding contract, not a semantic understanding claim.

For example, one UTF-8 source line contains the following token:

`urn:uuid:12345678-1234-1234-1234-123456789abc`

The witness records that WorkID, the byte offset of the first character of the token, and its byte length (45 bytes for this token). An editor that inserts a character before the token must regenerate the witness. This is intentionally mechanical: a source-level helper or build macro can insert the token, but v1 does not yet provide such editor integration, and hiding the token in a PDF would defeat this scanner.

A minimal release fragment is conceptually:

```
{
  "slot": {
    "line": "main",
    "version": 1,
    "work_id": "urn:uuid:...",
    "content": {
      "sha256": "...",
      "reference_work_ids": ["urn:uuid:..."],
      "citation_witnesses": [
        {
          "work_id": "urn:uuid:...",
          "byte_offset": 35,
          "byte_length": 45
        }
      ]
    }
  }
}
```

The ellipses above are presentation placeholders only. The released demo fixture binds a real manuscript's bytes, digest, and work identifier (the governance and PEC objects are real, not placeholders), using synthetic author identifiers. Real series releases are produced by the v1 fixture generator.

6 Series Layer and Cyclic Citations

6.1 Two graphs, two purposes

A paper remains verifiable after all series packages are removed. The optional series layer records relationships among independently complete releases. A signed series package contains

$$B_k = \text{Sign}_S(\text{seriesId}, \text{epoch}, \text{sequence}, \\ \text{previousBundleHash}, \text{rotationHash}, \text{rank}, \\ \text{releases}, \text{heads}, \text{semanticEdges}).$$

The semantic graph has vertices WorkIDs and may contain $P \rightarrow Q$ and $Q \rightarrow P$. Its edges express declared citation or related-work relations. The commitment graph has series anchors and completed ReleaseIDs. It does not make a release depend on a future series package. Thus two authors can cite stable random WorkIDs before both final documents are content-addressed; a later package resolves each WorkID to a completed ReleaseID. The semantic graph may be cyclic while the commitment graph remains a DAG.

The package includes a signed structural rank strictly above every release version it resolves. Combining anchor-to-release edges with exact release-to-parent edges decreases this rank along a commitment path. This rank is a graph certificate, not a timestamp, publication date, or priority claim.

The package fields have narrow roles: *epoch* names the active series key generation; *rotationHash* commits its authorized key transition; *heads* maps each WorkID to its advertised line head; and

semanticEdges records declared citation or related-work relations. The rank is only a finite measure certifying that the commitment graph has no cycle.

6.2 Revisions, branches, and conflicts

A revision names one exact predecessor. Multiple children of a parent are allowed, because a main manuscript and a computational appendix can be legitimate independent lines. A paper-level equivocation is instead detected when two valid, different releases occupy the same exclusive slot. A series-level equivocation is detected when two valid packages share the same series, epoch, sequence, and predecessor hash while committing to different contents.

An adversary can copy the content bytes, issue a fresh `WorkID` and keys, and obtain a valid new lineage. ACSD classifies this as content reuse under a different lineage, not as a cryptographic proof of plagiarism. That restraint is important: reuse may be authorized, collaborative, or independently derived.

As a concrete attack, suppose an attacker copies P-v1, changes its `WorkID`, and asks the series package to claim that it cites Q. The release verifier recomputes the content digest and byte witnesses; if the copied source does not contain the declared Q token, the package is rejected as an unsupported edge. If the attacker copies the token too, the package can still be valid under a new lineage. The evidence then says only “content reuse under a different lineage”; deciding plagiarism or authorization remains outside the verifier.

7 Scoped Post-release Statements

Research-process evidence. Version lineage and committed dialogue records can preserve complementary parts of a research history: a release fixes the manuscript at one stage, while a later opening exposes selected records already committed by that stage’s PEC. For example, authors may reveal the dialogue behind an early conditional result while keeping unrelated conversations private, then use a later version for a stronger result and its new supporting material. Exact event and release bindings prevent moving an opening to a different subject. They establish neither that the opened window is representative of the complete research process nor that contrary committed material was also opened. An author can therefore construct a selective but cryptographically valid narrative by revealing only favorable windows. Whether the disclosed reasoning scientifically supports the stated result, and whether omitted context changes that assessment, remain reviewable content questions. The current prototype supplies the commitment and disclosure checks; automatic capture of research history, AI-use declarations and contribution-to-event mappings remains incomplete.

Some evidence is necessarily published after the immutable release: authors may open a committed dialogue window or reveal one name after review. Treating these as arbitrary metadata would reintroduce policy amplification. ACSD uses a small common rule for a statement E of type k , exact subject s , signer a , and requested claim c :

$$\begin{aligned} \text{Grant}(E, c) \Rightarrow & \text{SignatureValid}(E) \\ & \wedge \text{ScopeValid}(E, s) \wedge \text{Authorized}(k, s, a) \\ & \wedge c \in \text{Permitted}(k) \wedge \text{Compatible}(k, c). \end{aligned}$$

This shared skeleton is intentionally not a general policy language. Event, identity, and time evidence retain different schemas, authorization principals, and proof predicates.

For a dialogue event, the verifier derives the required signer set from the already approved PEC rather than trusting identifiers listed by the disclosure. It then checks the disclosure policy, exact

PEC/event binding, contiguous Merkle opening, public-key identifiers, and every required COSE signature as one acceptance decision. A valid opening detached from its policy or a valid signature over a different opening grants nothing.

The slot-key identity assertion is an external per-slot sidecar binding the exact `ReleaseID`, `WorkID`, author slot, slot key, identity assertion, fixed purpose, and optional publication reference. Its result is deliberately named `SLOT_KEY_ASSENT_TO_IDENTITY_ASSERTION`: it says that the release slot’s pseudonymous key signed the mapping, not that a natural-person identity or venue acceptance was independently verified. In particular, possession of a stolen slot key is sufficient to create a protocol-valid assertion naming an uninvolved person. The verifier therefore exposes the narrow key-assent label and the unsupported natural-person claim; consumers must not render the result as authenticated civil identity without separate identity evidence. One slot can be revealed without revealing another. A full-byline result requires a valid sidecar for every release slot; conflicting assertions for one slot have no protocol- selected winner.

The aggregate verifier labels any proper valid subset `PARTIAL_BYLINE_KEY_ASSENT`; it returns `FULL_BYLINE_KEY_ASSENT` only after quantifying over every slot in the accepted release. An optional fail-incomplete mode lets a publication workflow require full coverage without denying that partial disclosure can be intentional. A common signed publication-reference string can crosswalk the released bytes to a later record, but it remains an author-key assertion: it does not derive venue acceptance. Extra contribution fields are rejected rather than reinterpreted as contribution evidence.

Selective disclosure is cryptographically scoped but not socially isolating. Revealing two members of a three-person team may permit inference about the remaining slot from public collaboration networks, so a partial byline is not an anonymity guarantee for undisclosed authors. Publication is also practically irreversible: neither a later contradictory sidecar nor a protocol status can make observers forget an identity string, and the signature alone cannot reveal whether assent resulted from error, coercion, or key compromise. Authors should therefore treat identity and dialogue disclosure as deliberate publication, not as a reversible account-setting operation.

Exact sidecar scope also does not create cross-paper unlinkability. Reusing the same public key in two unrelated `WorkIDs` exposes an equality edge even though a sidecar for one release cannot be replayed against the other. The read-only key-reuse audit reports this distinction; generating an independent key per unrelated lineage is the privacy-preserving default, not a proof against all traffic, account, or timing linkage.

7.1 Submission and unblinding lifecycle

A venue decision is deliberately not an ACSD state transition. Rejection can require no public action, leaving the previously released bytes unchanged. If review produces changed manuscript bytes, the submission becomes an exact predecessor-authorized successor. If the accepted text is byte-identical and only the byline is revealed, per-slot identity sidecars bind the existing release directly; manufacturing a redundant $n+1$ version would conflate identity disclosure with scientific revision. In either case, the authors may sign a publication reference, but only independently authenticated venue evidence could establish receipt or acceptance. Before publishing any sidecar, each participating author should inspect its exact release, slot and assertion; partial disclosure additionally requires a team-level privacy check because it may expose non-participating coauthors by inference.

8 Time Evidence

Time is intentionally external to author signing. An RFC 3161 request contains the SHA-256 imprint of the frozen approval set. A response supports the limited statement that the TSA signed a token for an imprint matching the locally recomputed complete target-and-signature set no later than its recorded `genTime`, only when the verifier supplies a TSA signer-certificate or fingerprint pin independently of the package. A certificate and its fingerprint copied into the same mutable package are not a trust anchor. The adapter checks the request/response nonce, SHA-256 imprint, `id-ct-TSTInfo` content type, single signer identifier, critical and exclusive timeStamping EKU, ESS certificate identifier, signed attributes, and CMS signature. It implements an exact-signer pin, not general PKIX path construction or revocation. Its local test TSA is a protocol test and can never produce `APPROVAL_SET_EXISTED_NOT_AFTER`.

There is no circularity: the timestamp receipt is a sidecar over the already closed approval set. A later revision can bind the receipt hash if a new signed object must refer to it. A timestamp does not prove creation at that moment, authorship, originality, or global first discovery. Multiple independently governed services can reduce single-service reliance, but their composition requires a separate policy.

Legacy releases timestamped only the approval target. They retain a distinct target-existence result and are never upgraded to an approval-set-time result. This typed distinction closes a temporal-confusion attack while preserving backward verification.

9 Implementation and Evaluation

The v1 prototype uses Python for deterministic fixture generation and a zero-dependency Node.js verifier for strict CBOR/COSE and profile validation; the two paths are intentionally separate. The package is designed for offline replay: locked dependencies are included, and the verification driver regenerates fixtures, verifies a standalone release, independently verifies the series profile, builds a manifest, and verifies that manifest.

A command-line tool (`acsd.py`) turns the profile into an executable workflow. Core commands are `keygen`, `init`, `author-approve`, `finalize`, and `verify`. Portable coauthor collection uses batch request/response directories and `coordinator-finalize`; the latter can exercise exact-target delegations addressed to supplied agent keys without receiving author private keys or requiring copied author identifiers. Lineage uses `authorize`, `recover`, and `revise`; a separate `compare-successors` command detects same-slot forks. Unblinding uses `disclose-identity` and `verify-identity`; the aggregate command `verify-identity-set` distinguishes partial from full coverage. Finally, `audit-key-reuse` checks cross-work public-key equality. A one-command `release` path covers a locally held key set. Approval status is derived from verified COSE files, never trusted from mutable state metadata. COSE uses the registered EdDSA value `-8`, requires tag 18, and rejects non-minimal or ambiguous CBOR encodings. A zero-dependency Node path independently verifies Python-produced approvals. The canonical JSON layer is exercised by 64 fixed Python–Node vectors plus 1,000 seeded generated cases; all generated cases agree on canonical bytes and text-layer verdicts. Three known object-layer float typing differences do not change the text-layer rejection result and remain recorded in the canonical specification. The installable wheel and immutable paper/evidence directory are separate artifacts; the authoritative gate runs generated work in temporary directories and confirms source-tree byte identity afterward.

The negative corpus includes the inherited rank, unsupported-edge, citation-offset, canonical-JSON, and path attacks. New cross-layer regressions replace a public key under an old key id;

Table 1: Typed evidence-to-claim confusion matrix. Checkmarks are the only derivable combinations; every off-diagonal cell is denied.

Evidence	Target time	Set time	Registered	Slot assent
Approval-target timestamp	✓	—	—	—
Approval-set timestamp	—	✓	—	—
SCITT inclusion	—	—	✓	—
Slot-identity assent	—	—	—	✓

coherently replace governance and PEC; recompute an unsigned target and attempt to reuse old approvals; alter COSE tags, algorithm identifiers, map keys, and lengths; replace a timestamp and rewrite the transport manifest; mismatch the nonce, signer, ECU, TSTInfo content type, or signature algorithm; and attempt to promote the local test TSA. All are rejected at their intended boundary. The single freeTSA result is an existence check that the adapter can consume one real service response, not a reliability study or a multi-provider conformance survey.

Additional regressions mutate approval-signature bytes after closure, attempt to relabel a target-only timestamp as approval-set time, detach a valid dialogue opening from its PEC or unanimous disclosure signatures, substitute a slot key, replay an identity mapping to another release, and promote a partial unblinding to a full byline. The deterministic demo now carries real Ed25519 release-approval and disclosure-approval signatures rather than self-reported signer identifiers. Its verifier reconstructs the release approval target, checks every required release approval, then verifies the separately scoped event disclosure against the accepted PEC digest.

The local private-key profile is deliberately separate from the release wire format. **keygen-encrypt** writes an encrypted PKCS#8 Ed25519 PEM using the maintained cryptographic library and prompts twice for a passphrase on an interactive terminal. All signing commands share one loader: no passphrase is accepted through an argument, environment variable, JSON value, or release artifact, noninteractive encrypted-key use fails closed, and a one-command release or revision prompts once per private-key path. The legacy plaintext PEM mode remains for compatibility, but is documented as a lower local-protection level rather than as a safe default. This protects a copied key file at rest; it neither proves key custody nor substitutes for an OS keystore, hardware signer, backup plan, or malware defense.

The executable semantic-confusion challenge applies four verified evidence kinds to four claim kinds. Only the diagonal combinations are derivable; all twelve cross-kind substitutions are denied. Three deliberately unsupported social claims—natural-person identity, originality, and contribution truth—remain unconstructible in the closed derivation table. A separate timeline confirms that a target timestamp obtained before key compromise does not establish that approvals added after compromise existed at that earlier time.

The lineage corpus begins with the smallest discriminating attack: a child with the exact WorkID and parent release/PEC digests, $n+1$, a fresh attacker key, and complete self-approval. It is rejected at the missing predecessor-quorum boundary. Positive cases cover unchanged authority, key rotation, one-of-two and two-of-two thresholds, and a named branch; negative cases cover threshold downgrade and transition-signature replay. Two valid same-slot siblings yield equivocation with no winner.

The recovery corpus begins from a parent that committed two disjoint guardians with a two-of-two threshold. Their signatures authorize one replacement-author child only because the child changes online authority and every child author approves its exact target. The runner rejects an uncommitted guardian, a parent without a recovery commitment, an insufficient guardian quorum, a recovery signature replayed to different child bytes, mixed online and recovery methods,

and recovery used merely to alter recovery configuration. A separately ordinary- authorized and recovery-authorized child in the same slot remains a reported fork with no winner. Thus recovery restores conditional liveness without claiming guardian custody, retroactive revocation, complete fork visibility, or an offline global head.

9.1 Scripted distributed-author workflow

We additionally execute the production CLI as four isolated roles: three authors and one coordinator agent. The coordinator initializes an ordered three-author candidate and exports one self-contained request per slot. Two authors return direct approvals; the third signs a non-redelegable capability for the exact target and the distinct coordinator-agent key. One transactional `coordinator-finalize` invocation imports all responses, discovers the delegated slot from that agent key, exercises the capability, and finalizes the candidate. Offline verification then derives the narrower authorized-target approval outcome for the mixed set.

After key setup, the happy path comprises seven CLI invocations: initialization, batch export, three author responses, coordinator finalization, and verification. It requires no JSON edits, no private-key transfer, and no manual copying of a digest, WorkID, or key identifier. In an adverse run, the final returned signature is changed and its transport manifest regenerated. Batch import rejects the collection and leaves the live candidate with zero imported approvals. The runner contacts no external service and uses an explicit untimestamped test-only finalization policy.

This is a scripted role-based integration experiment with zero human participants and zero production deployments. It demonstrates executable role separation and transactional failure semantics; it does not establish first-time human usability, adoption, or safe key-backup behavior.

To avoid a weak comparison, a controlled mechanism baseline fixes the signature primitive to Ed25519 and evaluates three policies over the same parent and $n+1$ candidates. Child-declared-key verification accepts both legitimate rotation and fresh-key capture; predecessor-key pinning rejects both. A manual predecessor signature over exact parent, child, and successor-key bytes correctly accepts the rotation and rejects replay onto the attacker child. ACSD reaches the same core decision. The positive control isolates the effect of explicit predecessor authorization. The surrounding canonical object, threshold governance, approval closure and typed claim rules specify the remaining application contract.

Three further executable workflows test post-release use. A two-author submission lifecycle leaves a rejected release byte-identical, creates an authorized successor only when manuscript bytes change, and binds an unchanged release to a later publication reference without deriving acceptance. A three-author unblinding run distinguishes 1/3 and 2/3 partial disclosure from 3/3 full coverage, and rejects duplicate slots, name mutation, and a smuggled contribution field. A cross-paper run rejects sidecar replay across releases while reporting that reuse of the same public key makes two WorkIDs publicly linkable.

9.2 Actual-client cost and security comparison

A separate local experiment compares the production ACSD CLI, an independent minimal detached-Ed25519 client, and GnuPG 2.4.9 (libcrypt 1.12.2). The protocol fixes 64 KiB, 1 MiB and 8 MiB manuscripts and one or three authors. Each of the 18 configurations receives one warm-up and five measured repetitions, with shuffled recipe order: 90 measurements and 18 warm-ups. Timings include process startup and file I/O. Key generation and verifier key import are excluded setup operations. Both private and public keyrings are isolated test directories; GPG verification checks each required signer fingerprint.

Table 2: Executable evaluation summary. The artifact retains the complete case-level report, commands, and expected outcomes.

Evidence family	Count / result
Legacy release/series profile	8 releases, 18 endorsements, 7 series objects; 30/30 checks
Canonicalization and manifests	64 fixed plus 1000 generated differential cases; 113 entries
Implementation regression suite	195 tests discovered; 3 conditional skips locally
Authorization and recovery	8 lineage cases; 6 recovery rejection classes plus an unranked fork
Post-release workflows	submission lifecycle, 1/3–3/3 unblinding, and cross-work isolation
Operational controls	scripted three-author run; manual exact-transition positive control
External-time interoperability	one real freeTSA response verified offline
Executable formal boundary	35 scoped, 15 lineage, and 19 appraisal Python–Lean decisions
Mechanized model	119 theorems; no <code>sorry</code> / <code>admit</code>

Table 3 shows the cost of the declared recipes. For one author, verification sample standard deviations are 44, 15 and 58 ms for detached Ed25519, GPG and ACSD, respectively; for three authors they are 39, 22 and 96 ms. The complete raw report retains every sample, all sizes, minima, maxima, standard deviations, commands and source hashes. GPG packaging uses one signing and one public-key export command per author; batching exports or changing packaging could reduce that cost. A wrapper’s single command does not measure human coordination effort. Ordinary host activity and nonmonotonic size trends preclude a throughput or scaling claim.

The overhead is the price of application semantics, not stronger signature bits. For the one-author 1 MiB case, ACSD takes about 1.8 times the detached Ed25519 creation time, 1.4 times its verification time, and stores about 6.1 KiB more metadata. With three authors, the corresponding ACSD metadata is about 10.7 KiB, roughly twenty times the minimal detached-signature sidecars, while median creation and verification remain below one second on this host. This trade is appropriate when a verifier must later decide team assent, exact successor authority, recovery, completed-set time, or scoped disclosure from a maintained package. For a one-off claim that selected keys signed immutable bytes, detached signatures plus independently chosen time or archive evidence are simpler and sufficient; ACSD should not be used merely to reproduce that narrow statement.

Actual GPG checks accept the complete three-signer case, reject manuscript mutation and a missing signature, and distinguish a self-declared attacker key from a pinned predecessor key. A GPG-signed manual transition validates successfully and rejects altered child binding. That test is a statement-binding control; the Ed25519 baseline additionally reconstructs the transition from the candidate child. Lifecycle, partial-disclosure and cross-release replay cases were rerun for ACSD.

Recipe	Authors	Create (ms)	Verify (ms)	Extra bytes	Files
Detached Ed25519	1	257	262	177	3
GnuPG	1	245	81	348	3
ACSD	1	464	362	6,284	11
Detached Ed25519	3	228	214	531	7
GnuPG	3	682	252	1,044	7
ACSD	3	480	341	10,934	15

Table 3: Median local costs for a 1 MiB manuscript. Extra bytes exclude the manuscript. Recipes verify different application relations; this is not a ranking at equal security.

Attack	Proved property
Reuse an approval	Approval binds its exact target; a different target cannot reuse it.
Capture the next version	Accepted succession has predecessor authorization or the specified precommitted recovery authorization.
Replay a transition	Authorization binds the exact child and cannot authorize another child.
Inflate a partial reveal	Full-byline assent requires every required slot.
Reuse evidence across scopes	Derived claims require compatible evidence for the exact subject.
Backdate completed assent	Target-only time evidence cannot derive approval-set time.

Table 4: Selected Lean properties. They concern protocol authorization and claim composition under the model’s declared evidence assumptions.

They test additional application semantics, rather than defects in the underlying GPG signature algorithm.

The same real 4,649-byte RFC 3161 receipt was also verified in two offline paths, each with a warm-up and five measurements. Receipt-only verification had median 272 ms (SD 16); checking the ACSD approval-set binding as well had median 295 ms (SD 28). Both use the same receipt parser and pinned signer; these are incremental time-evidence checks and exclude manuscript-signature verification. Sixteen associated receipt and approval-closure tests passed. No request was sent to a service during this experiment. OpenTimestamps confirmation, archival service latency, independent users and clean-machine replication remain unmeasured.

The earlier in-memory measurements remain prototype-scale rather than a throughput claim. They were recorded under Python 3.12.14 on 64-bit Windows 10.0.26200, a 13th Gen Intel Core i7-1360P with 16 logical processors, and 31.7 GiB physical memory. In that run, five repetitions of an in-memory workload that canonicalizes objects, constructs and verifies SHA-256 manifest entries, and scans a linear rank certificate had median times of 2.23, 26.45, 227.54, and 718.81 ms for 10, 100, 1,000, and 5,000 objects respectively; peak traced memory at 5,000 objects was 7.04 MiB. This excludes filesystem, COSE, and network cost, so it is a smoke test for obvious scaling anomalies, not a statistically powered performance evaluation; dispersion beyond minimum and median was not retained.

9.3 Formal security properties

Table 4 states the main attack-exclusion claims. The applicable ordinary or recovery quorum and already verified adapter facts are model inputs. Proofs rule out the stated attacks for all modeled instances satisfying those assumptions.

The current Lean development has 119 theorem declarations over approval targets, approvals,

digest-linked events, a closed outcome type, exact-subject scoped statements, lineage transitions, recovery transitions, and claim-free appraisal transcripts. They prove, among other facts, that an approval for one target digest cannot be reused for a different target; every granted result has policy permission and compatible exact-subject evidence; all required approvals are present; and external time requires the approval-set subject, signer pin, nonce, RFC profile, and a non-local authority. The lineage theorems show that every accepted successor has the applicable authority evidence, a fresh authority cannot enter through the unchanged-authority path, and a transition bound to one child digest cannot authorize another child. The recovery theorems require a predecessor commitment, disjoint recovery authority, changed online authority, and an exact transition, while preserving the fact that a distinct ordinary-authorized edge is not retrospectively erased. The scoped layer proves that identity evidence cannot grant an event-opening claim, a missing slot prevents full-byline status, and a target-only timestamp cannot establish approval-set time. Evidence-set union cannot manufacture a capability: every derived claim still has compatible verified support in one component.

Unlike the earlier purely abstract boundary, Lean now strictly decodes the restricted canonical JSON for the maintained claim-free certificates and the generic `acsd-appraisal-transcript/v1` used by `acsd verify`. Its executable checker constructs exact-subject derivations, and refinement theorems map those derivations to the earlier digest-scoped abstraction for any chosen subject projection. Python and Lean agree on 35 scoped-certificate, 15 authorized-lineage, and 19 production-appraisal positive and adverse cases. The model deliberately accepts already checked signature, digest, and adapter facts as inputs: it does not refine filesystem loading, release-object parsing, DER/CBOR or COSE adapters, RFC 3161 processing, or the cryptographic primitives, and it does not prove TSA honesty or general PKIX correctness.

The formal tree also includes a bounded multi-premise experiment: one abstract RATS-style appraisal result requires exact token/attester, measurement, nonce, reference-value, and verifier-policy facts. It is a proof that the small permission-plus-exact-premises pattern admits an independently meaningful second instance. A finite subject-bound profile also proves that a permitted conclusion has subject-matched support when every rule has nonempty, subject-matched premises. These are proof-engineering experiments; the ACSD instance retains its stronger typed appraisal schema. They do not implement EAT parsing or a deployed attestation service.

The source contains no `sorry`, `admit`, or project-specific `axiom` declaration. The dependency audit reports Lean’s standard logical principles—`propext`, `Quot.sound`, and `Classical.choice`—for the decoded and extensional results that use them; these are not claims of a constructive or axiom-free development.

These 119 are PEC, lineage, recovery, scoped-appraisal, transcript, cross-layer composition, and bounded-generalization theorems, not a recount of the separately archived v1 Lean core. The v1 artifact records 53 theorem statements about release catalogs, teams, lineage, series graphs, citation witnesses, and lifecycle rules. That earlier model remains an inherited, distinct proof surface; it was neither deleted nor compressed into the new count. The current count concerns the executable composition model of approval-target reuse, capability gating, event links, typed external-time preconditions, selective unblinding, authorized succession and recovery, strict appraisal-transcript checking, and the bounded generalization experiment.

10 Related Work

The profile builds on established foundations. PROV-DM supplies a general vocabulary for entities, activities, agents, derivations, and bundles [1]. Software Heritage identifiers provide intrinsic

identifiers for software objects [2]; Trusty URIs provide hash-bearing immutable artifact identifiers [3]. COSE defines the signed binary envelope used for approvals [4]. SCITT specifies a broader architecture for signed statements and transparency services [5], while COSE Receipts standardizes receipt envelopes [6]; neither registration nor a valid receipt makes the registered statement true. RFC 3161 specifies the timestamp protocol [7]; RFC 9921 explicitly distinguishes COSE-then-timestamp from timestamp-then-COSE [8]. ACSD uses the former ordering as a semantic analogy: its separate approval-set object is timestamped after signature collection, without claiming wire-level interoperability with the RFC 9921 header profile. TUF uses predecessor and successor root thresholds to preserve trust through root-key updates [15]; in-toto binds supply-chain steps to authorized functionaries [16]. ACSD specializes related authorization ideas to an exact anonymous scholarly WorkID succession rule.

C2PA Content Credentials bind signed provenance assertions to media assets and explicitly separate tamper validation from judgments about whether an assertion is good or true [11]. The CAWG Identity Assertion adds creator identity assertions and validation results for that ecosystem [12]. ClaimChain provides privacy-preserving authenticated claim repositories with non-equivocation goals [13]. These systems establish signed provenance, scoped identity, selective disclosure, and conflict resistance as existing design families. ACSD contributes a smaller scholarly-release profile that evaluates approval completion, revision authority, mutual-citation separation, a closed claim algebra, and executable counterexamples together.

SD-JWT defines selective disclosure for claims in a signed JSON payload [9], while BBS-based verifiable-credential suites additionally target unlinkable derived proofs [10]. For public linkage, ACSD instead publishes a direct link from an already public release-slot key to one identity assertion. This smaller mechanism avoids an issuer/holder credential layer, while the typed claim boundary leaves unlinkable credentials as a separable service level.

A detached signature already authenticates exact bytes under a chosen public key. Moreover, a minimal predecessor-signed transition over exact parent, child, and successor-key bytes can reproduce ACSD’s core key-rotation decision. Our controlled baseline confirms this positive result instead of comparing only against self-declared or permanently pinned keys. ACSD’s distinction is the integration of this edge with canonical scholarly objects, ordered and threshold author authority, signature-complete closure, scoped disclosures, and a closed output vocabulary.

ACSD combines these established mechanisms into an executable scholarly profile: self-contained anonymous releases; exact predecessor-authorized succession; a separate series layer for mutual citations; exact text-token witnesses; explicit conflict predicates; and counterexamples against common overclaims about signatures, local clocks, and copied content. Appendix A summarizes the nearest-neighbor capability boundary.

11 Use of AI in the Research Process

Generative AI assistants were used in protocol exploration, adversarial design review, implementation, test development, manuscript drafting and revision, and preparation of figures. These uses affected research design and software artifacts, not merely prose editing. Human authors remain responsible for the claims, source selection and interpretation. Reported claims are tied to machine-executed tests, retained experiment outputs, independent implementation comparisons and Lean checking; those checks reduce specific error classes but do not substitute for human scientific review. No experiment in this article is presented as a study of human participants or as evidence of independent human usability.

12 Limitations and Future Work

Several extensions are intentionally deferred. First, relations across independently administered series require policy and an interoperable cross-series package model. Second, anti-equivocation evidence from transparency logs or gossip requires deployed service assumptions. Third, although Lean strictly decodes the restricted appraisal transcript, the filesystem, release-object, DER/CBOR, COSE, RFC 3161, canonicalization, hash, and signature adapters remain an explicit trusted implementation boundary rather than proof targets for this profile. Fourth, encrypted PKCS#8 protects an exported key file at rest, but operating-system-backed storage, hardware signing, backup/migration workflows, and malware resistance remain distinct service levels. Fifth, exact TSA signer pinning does not replace PKIX path construction or revocation when a deployment requires those services. Long-lived evidence also needs an explicit renewal policy, such as renewed timestamps or an evidence-record scheme when algorithms or certificates age [17]. Sixth, precommitted recovery avoids post-hoc takeover but does not prove guardian custody, revoke a valid historical transition, discover withheld competing children, or select a global head without transparency, pinning, gossip, or governance. Seventh, the actual-client comparison measures local GnuPG, detached Ed25519 and ACSD creation/verification costs; it does not measure OpenTimestamps or archival services, independent users, or production deployment. Eighth, scoped disclosures provide integrity and exact binding, not authenticated natural-person identity, freedom from coercion, completeness of a selectively opened research record, or resistance to social inference about undisclosed collaborators. A higher-assurance identity service could require an independently authenticated subject countersignature or credential; we deliberately exclude the resulting issuer and revocation trust model from this profile. Finally, content-anonymous publication does not hide repository-account, network, timing, or public-key linkage, and richer citation/editor integration remains future work.

These are reasons to state the present result narrowly, not reasons to weaken the useful evidence it already preserves.

13 Conclusion

ACSD turns a vague “anonymous signed paper” idea into an executable evidence profile. It binds exact objects to pseudonymous team approval and composes predecessor-authorized succession, exact recovery, mutual citations, fork detection, and scoped disclosures under a closed claim vocabulary. Ordinary primitives supply individual decisions; ACSD contributes their scholarly composition and modeled attack exclusions. Natural-person identity, originality, global head selection, and independent time remain separate claims.

References

- [1] W3C. *PROV-DM: The PROV Data Model*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-dm/>
- [2] Software Heritage. *SWHID Specification: Core Identifiers*, version 1.2. https://www.swhid.org/specification/v1.2/5.Core_identifiers/
- [3] T. Kuhn and M. Dumontier. *Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data*. ESWC, 2014. https://doi.org/10.1007/978-3-319-07443-6_27
- [4] J. Schaad. *CBOR Object Signing and Encryption (COSE): Structures and Process*. RFC 9052, 2022. <https://www.rfc-editor.org/rfc/rfc9052.html>

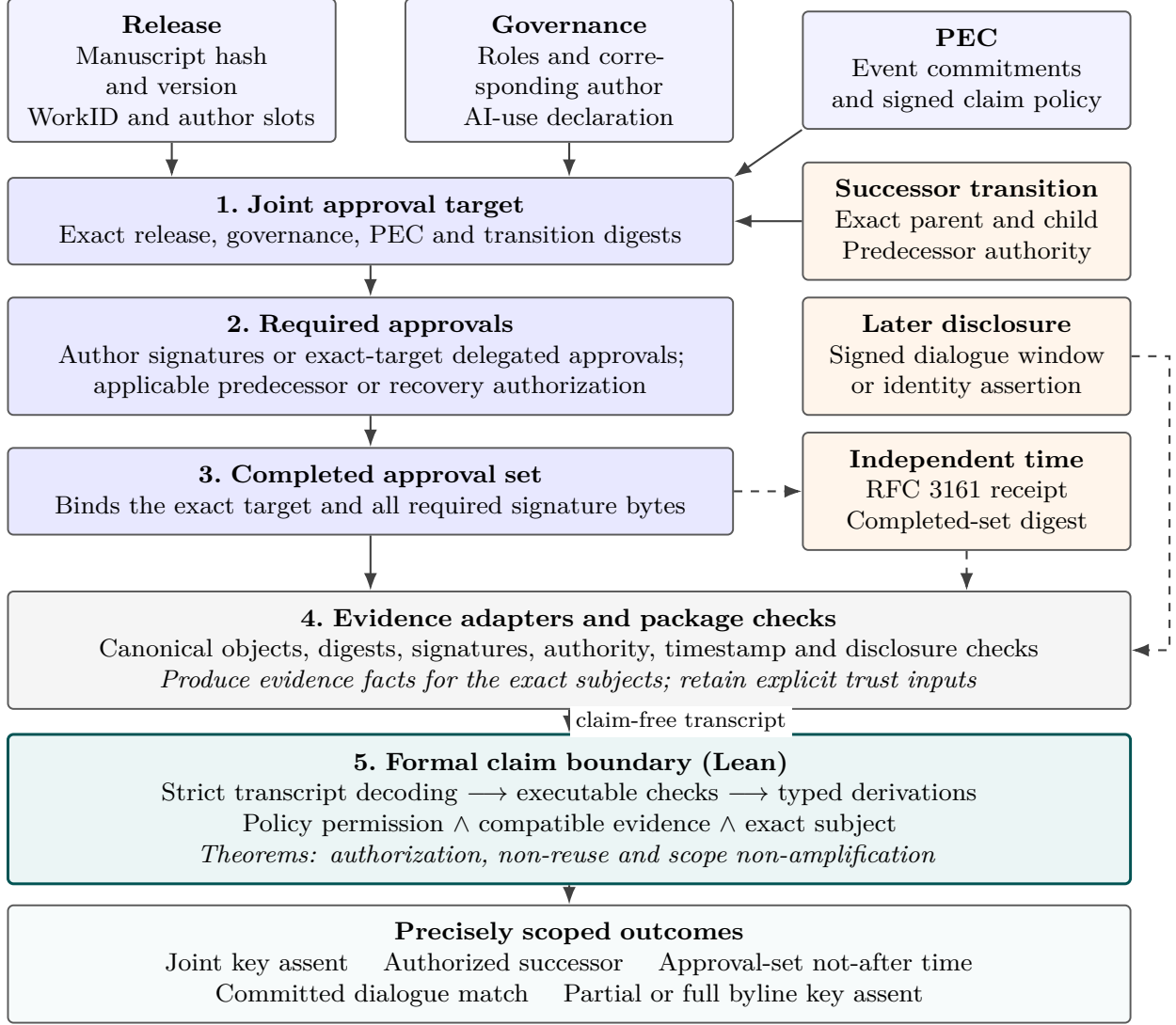
- [5] H. Birkholz et al. *An Architecture for Trustworthy and Transparent Digital Supply Chains*. RFC 9943, 2026. <https://www.rfc-editor.org/rfc/rfc9943.html>
- [6] O. Steele et al. *CBOR Object Signing and Encryption (COSE) Receipts*. RFC 9942, 2026. <https://www.rfc-editor.org/rfc/rfc9942.html>
- [7] C. Adams et al. *Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)*. RFC 3161, 2001. <https://www.rfc-editor.org/rfc/rfc3161.html>
- [8] H. Birkholz, T. Fossati, and M. Riechert. *CBOR Object Signing and Encryption (COSE) Header Parameter for Timestamp Tokens as Defined in RFC 3161*. RFC 9921, 2026. <https://www.rfc-editor.org/rfc/rfc9921.html>
- [9] D. Fett, K. Yasuda, and B. Campbell. *Selective Disclosure for JSON Web Tokens*. RFC 9901, 2025. <https://www.rfc-editor.org/rfc/rfc9901.html>
- [10] W3C. *Data Integrity BBS Cryptosuites v1.0*. Candidate Recommendation Draft, 2026. <https://www.w3.org/TR/vc-di-bbs/>
- [11] Coalition for Content Provenance and Authenticity. *Content Credentials Technical Specification*, version 2.4, 2026. https://spec.c2pa.org/specifications/specifications/2.4/specs/C2PA_Specification.html
- [12] Creator Assertions Working Group. *Identity Assertion*, version 1.3, 2026. <https://cawg.io/identity/1.3/>
- [13] B. Kulynych, W. Lueks, M. Isaakidis, G. Danezis, and C. Troncoso. *ClaimChain: Improving the Security and Privacy of In-band Key Distribution for Messaging*. WPES, 2018. <https://doi.org/10.1145/3267323.3268947>
- [14] OpenTimestamps. *A timestamp proof standard for blockchain anchoring*. Project documentation, accessed 2026-09-11. <https://opentimestamps.org/>
- [15] The Update Framework. *The Update Framework Specification*, version 1.0.28. <https://theupdateframework.github.io/specification/v1.0.28/>
- [16] in-toto. *in-toto Specification*. <https://github.com/in-toto/docs/blob/master/in-toto-spec.md>
- [17] T. Gondrom, R. Brandner, and U. Pordesch. *Evidence Record Syntax (ERS)*. RFC 4998, 2007. <https://www.rfc-editor.org/rfc/rfc4998.html>

A Nearest-neighbor capability boundary

Table 5 is a capability map, not a performance ranking. Each neighboring system remains preferable when only its native claim is required.

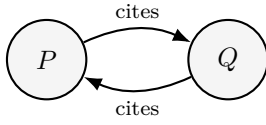
Table 5: Nearest-neighbor capability boundary.

System	Primary capability	ACSD boundary
Detached signature + exact transition	Exact-byte key control and hand-built successor authorization	Matches the core edge decision; application must define closure, thresholds, scopes, and claims
OpenTimestamps RFC 3161	Independent timestamp evidence	Establishes a committed imprint’s time under its trust model, not author roles or succession
Zenodo / SWHID	Versioned archival records and persistent intrinsic identifiers	Archives or identifies objects; ACSD binds pseudonymous team assent and authorized evolution
TUF / in-toto	Threshold key update and authorized supply-chain steps	Closest authorization analogue; ACSD specializes the decision to scholarly releases and disclosure claims
C2PA / CAWG / SD-JWT / BBS / Claim-Chain	Provenance, identity assertions, selective or privacy-preserving claims	Broader claim ecosystems; ACSD uses public slot sidecars and a closed scholarly claim boundary
ACSD	Release, governance, revision, witness and disclosure profile	Composes these decisions while retaining each backend’s trust assumptions

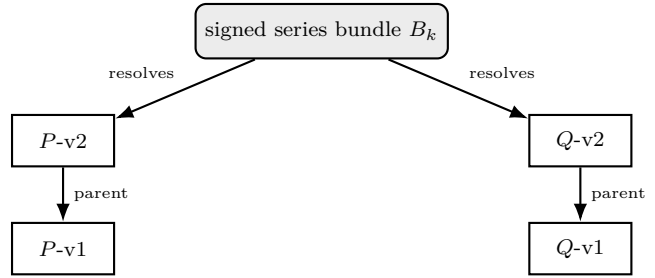


Solid arrows: construction or verification flow. Dashed arrows: optional time/disclosure paths.
The optional series layer resolves completed releases; it does not gate single-paper verification.

Figure 1: ACSD architecture from committed objects to justified claims. Successors use an exact predecessor-authorized transition; genesis has no transition. The completed approval set is the timestamp subject. Later disclosures retain their original PEC/event or release/slot scope. The Lean boundary derives claims above checked adapter facts. Diagram arrows show data flow. They are distinct from the commitment arrows in Figure 2.



(a) Semantic graph: cycles allowed



(b) Commitment graph: a DAG

Figure 2: ACSD separates cyclic semantic citations over stable WorkIDs (a) from acyclic exact-byte commitments through release digests and predecessor links (b). Commitment arrows point toward the committed object and indicate neither chronology nor priority.